UART Hardware and Pin Functions

ECE 2564, Embedded Systems

Fall 2025

Require Resources for this Module

- Download and install **MobaXterm**
- Import **this** repository

UART Reminders

UART reminders

- Universal Asynchronous Receiver/Transmitter
- A UART is a device that handles the data transmit and receive functions of an *asynchronous* serial communication channel
- A UART converts data from parallel to serial for transmission, and from *serial* to *parallel* on reception
- UARTs may provide additional signals that indicate the state of the transmission media
- UARTs can *regulate* the flow of data (in some cases), which is useful in situations where the remote device is not prepared to accept more data

UART Tx/Rx

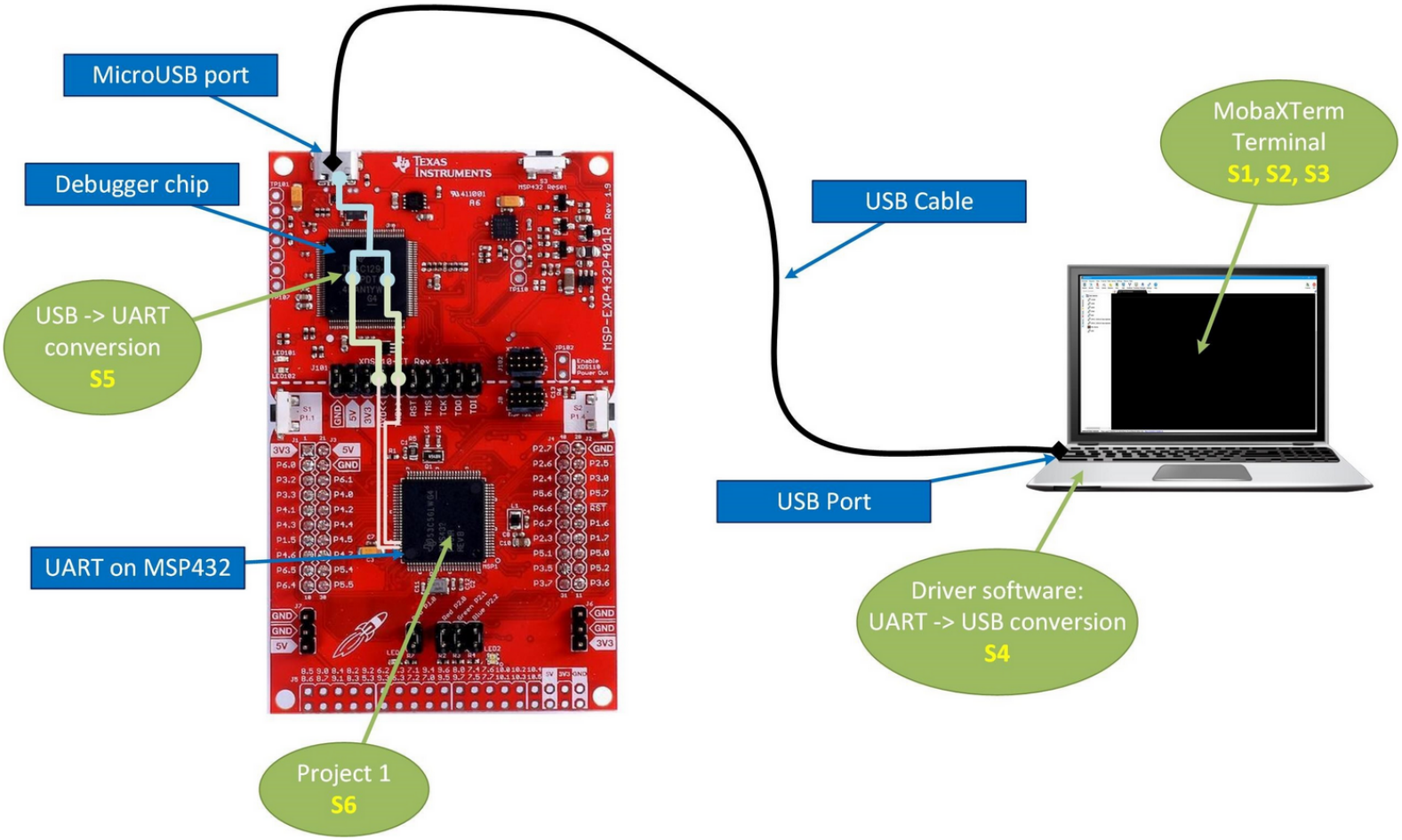
- A UART typically operates on computer logic voltage levels
- Serial interfaces to other devices often require *line drivers*
  - A line driver is a circuit the provides the needed voltage, current and signal conditioning to transmit digital data reliably over a distance
- RS-232 is an example of an asynchronous serial standard



Line drivers inserted between UARTs

What Goes on with the UART for our Setup

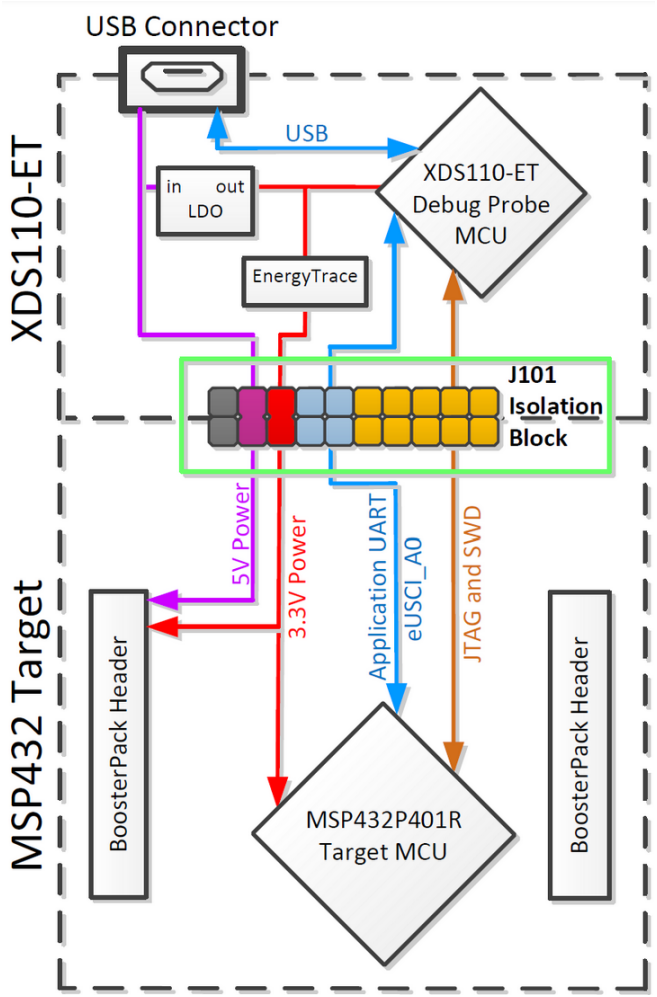
- The green bubbules represent software operations
- The blue boxes represent hardware operations



Getting from your computer to the MSP432 via UART

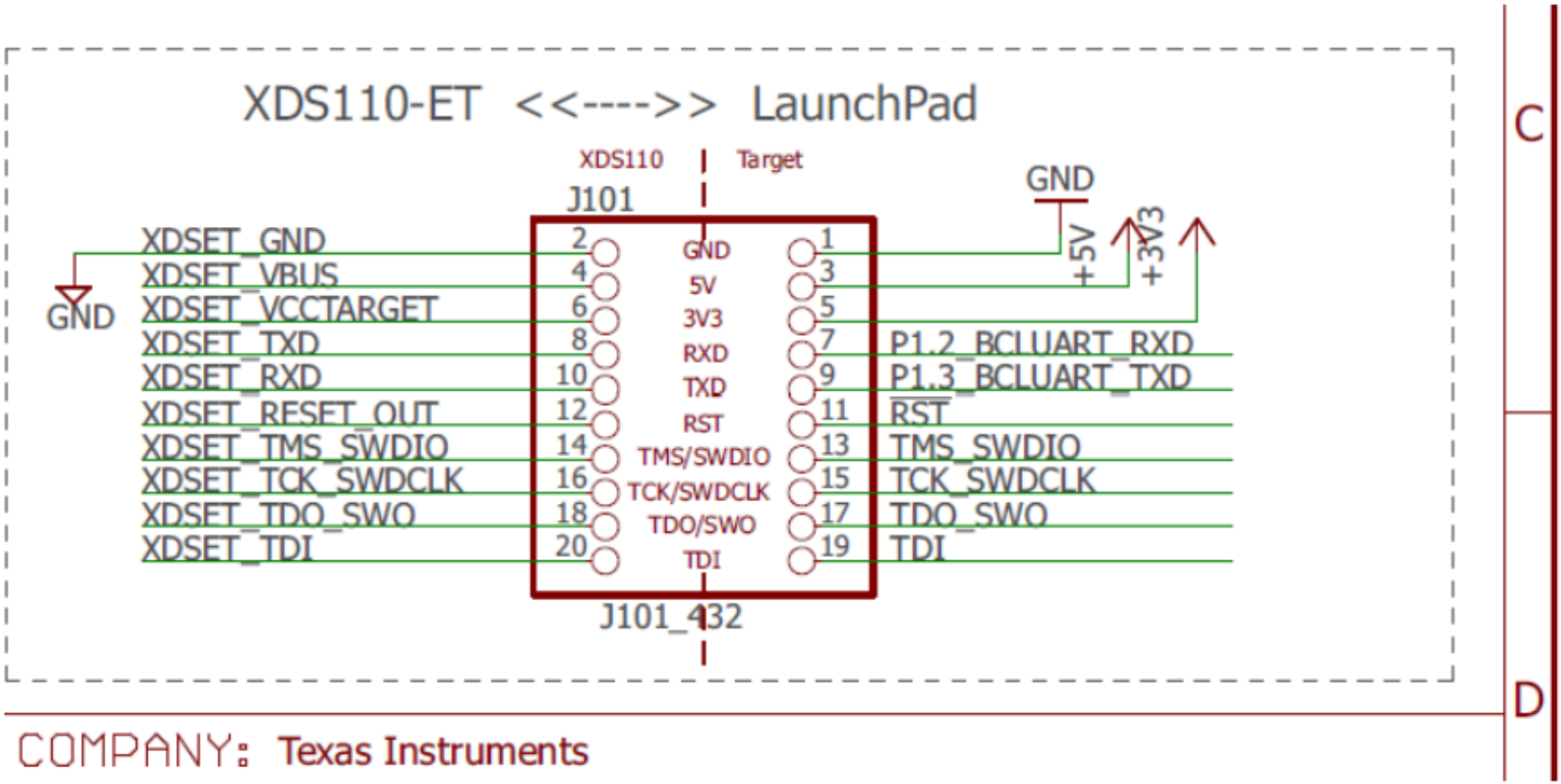
- S1: MobaXTerm reads a character from keyboard
- S2: MobaXTerm creates a packet of bits that consists of the UART packet (frame)
- S3 MobaXTerm sends the packet to what it thinks is a serial port
- S4: Another software (the driver) updates the packet to a form that can go through a USB port
- S5: A software running on the debugger chip unpacks the packet, which is in a USB format, and creates a UART packet of data which is identical to what was created in step 2
- S6: The UART module samples this serial signal and records the data bits in one of its registers. The ARM CPU reads this register using a load instruction.

Launchpad Overview of USB and UART Connections[1]



[1] From LaunchPad User Manual - Page 8

UART and the J101 Connector[2]



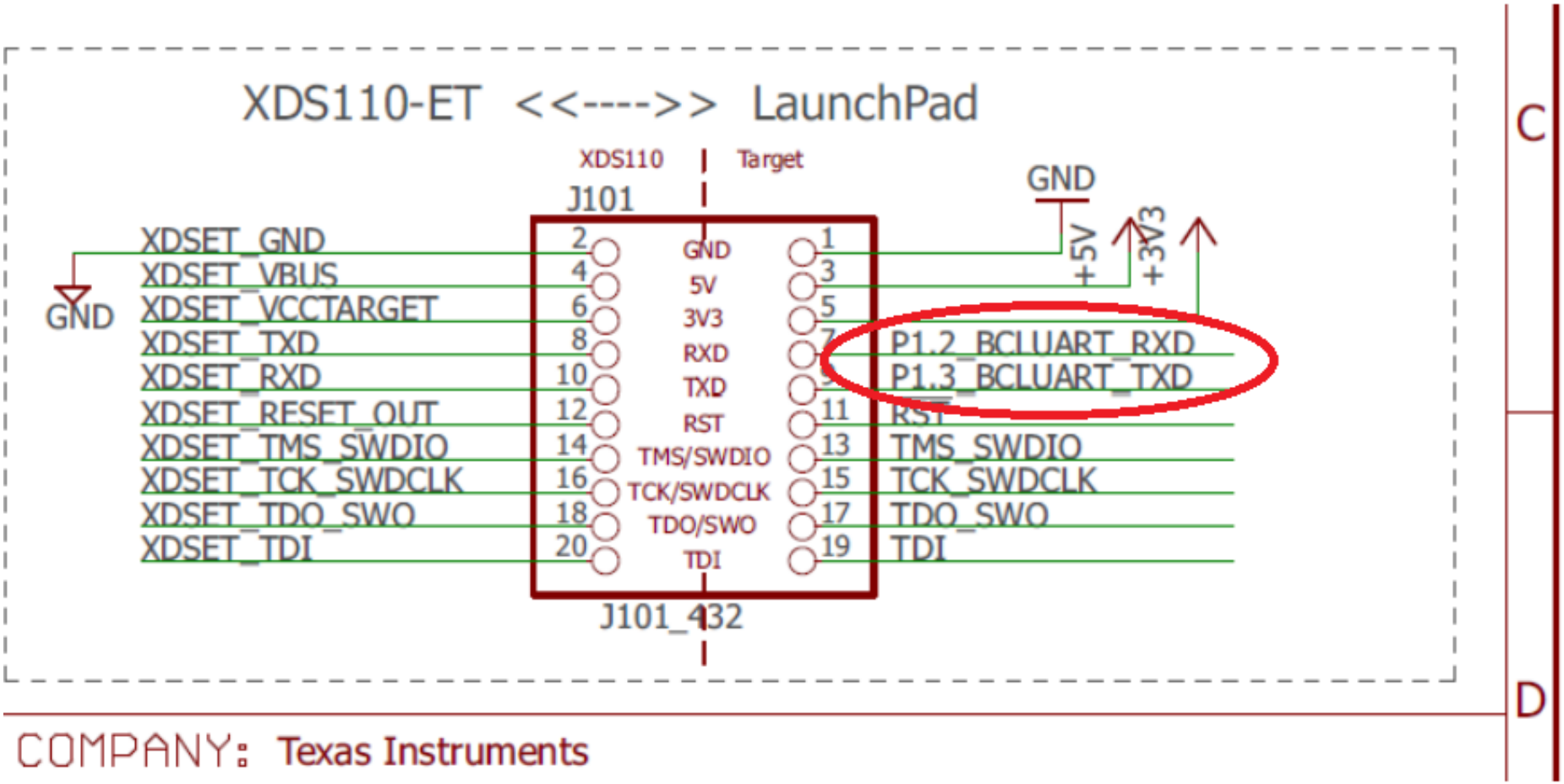
[2]From LaunchPad User Manual - Page 38

MSP432 Port/Pin Connection for UART Connection

Which MSP432 port/pins are connected to J101 connectors that can be used as Rx and Tx signals for UART communication with devices outside of the LaunchPad?

Launchpad Overview of USB and UART Connections

P1.2: Rx; P1.3: Tx



Pin Function Tables

- There are a number of tables in the MP432 datasheet that cover pin functionality
- Look at pages 138–174
- To learn about a pin, search for the pin name in the datasheet

Port 1 Pin Functions[3]

What UART functions can Pin x of Port1 perform? What should P1SEL0 and P1SEL1 be?

- Pin1
- Pin2
- Pin3

PIN NAME (P1.x)	x	FUNCTION	CONTROL BITS OR SIGNALS[1]		
			P1DIR.x	P1SEL1.x	P1SEL0.x
P1.0/UCA0STE	0	P1.0 (I/O)	I: 0; O: 1	0	0
		UCA0STE	X[2]	0	1
		N/A	0	1	0
		DVSS	1		
		N/A	0	1	1
		DVSS	1		
P1.1/UCA0CLK	1	P1.1 (I/O)	I: 0; O: 1	0	0
		UCA0CLK	X[2]	0	1
		N/A	0	1	0
		DVSS	1		
		N/A	0	1	1
		DVSS	1		
P1.2/UCA0RXD/UCA0SOMI	2	P1.2 (I/O)	I: 0; O: 1	0	0
		UCA0RXD/UCA0SOMI	X[2]	0	1
		N/A	0	1	0
		DVSS	1		
		N/A	0	1	1
		DVSS	1		
P1.3/UCA0TXD/UCA0SIMO	3	P1.3 (I/O)	I: 0; O: 1	0	0
		UCA0TXD/UCA0SIMO	X[2]	0	1
		N/A	0	1	0
		DVSS	1		
		N/A	0	1	1
		DVSS	1		
P1.4/UCB0STE	4	P1.4 (I/O)	I: 0; O: 1	0	0
		UCB0STE	X[3]	0	1
		N/A	0	1	0
		DVSS	1		
		N/A	0	1	1
		DVSS	1		
P1.5/UCB0CLK	5	P1.5 (I/O)	I: 0; O: 1	0	0
		UCB0CLK	X[3]	0	1
		N/A	0	1	0
		DVSS	1		
		N/A	0	1	1
		DVSS	1		
P1.6/UCB0SIMO/UCB0SDA	6	P1.6 (I/O)	I: 0; O: 1	0	0
		UCB0SIMO/UCB0SDA	X[3]	0	1
		N/A	0	1	0
		DVSS	1		
		N/A	0	1	1
		DVSS	1		

(1) X = don't care
(2) Direction controlled by eUSCI_A0 module.
(3) Direction controlled by eUSCI_B0 module.

[3] Datasheet Table 6-62

Port 1 Pin Functions

				SEL1	SEL0
P1.1/UCA0CLK	1	P1.1 (I/O)	I: 0; O: 1	0	0
		UCA0CLK	X(2)	0	1
		N/A	0	1	0
		DVSS	1		
		N/A	0	1	1
		DVSS	1		
P1.2/UCA0RXD/UCA0SOMI	2	P1.2 (I/O)	I: 0; O: 1	0	0
		UCA0RXD/UCA0SOMI	X(2)	0	1
		N/A	0	1	0
		DVSS	1		
		N/A	0	1	1
		DVSS	1		

P1.1: Related to Clock

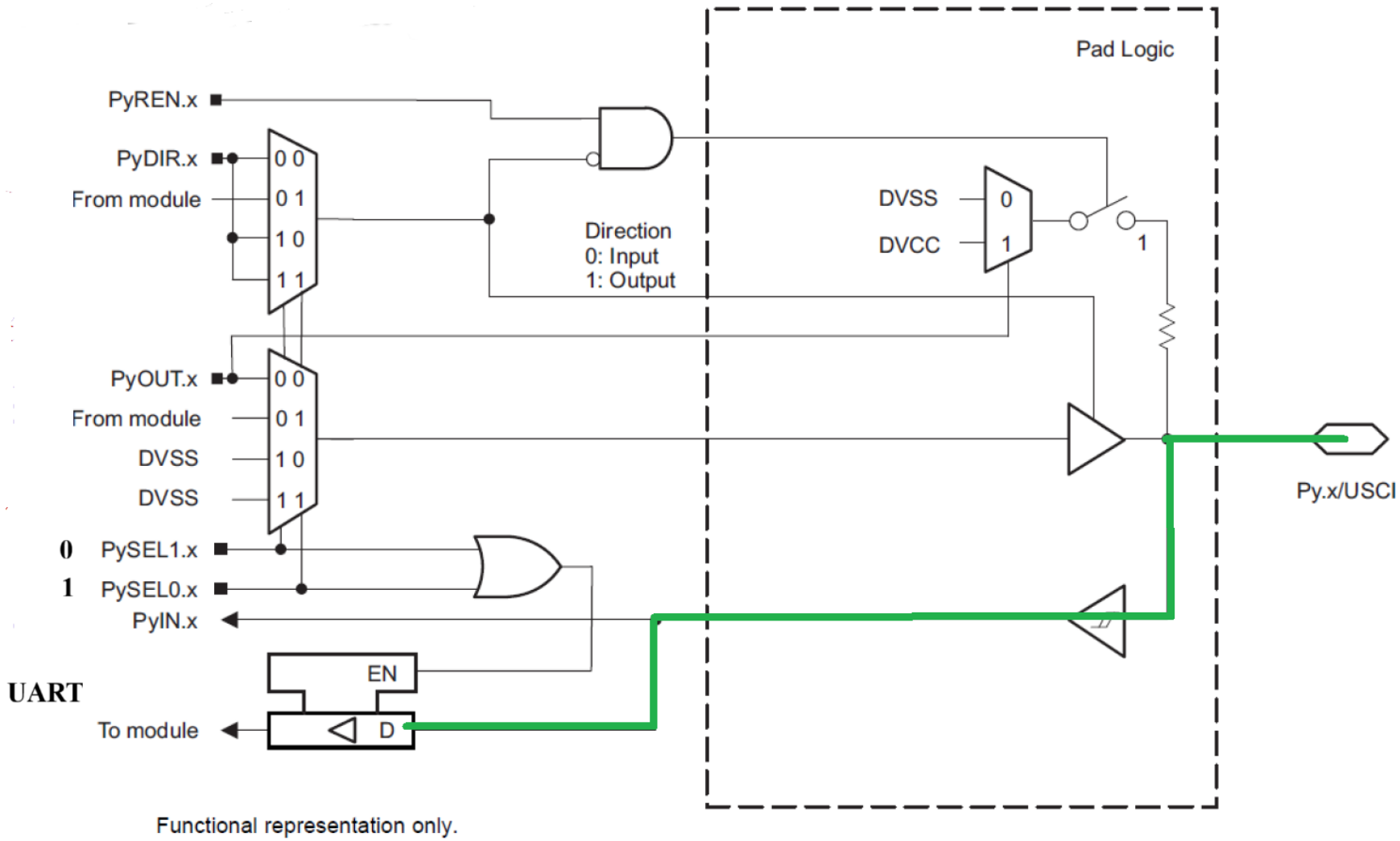
P1.2: Related to Rx

P1Sel1 and P1Sel2

- 00: Digital I/O pin (P1Dir = 0 or 1)
- 01: Rx Tx (P1Dir = X)
  - UCA0RXD (UART) / UCA0SOMI (SPI)
  - Note: UART and SPI use the same pins for Tx/Rx

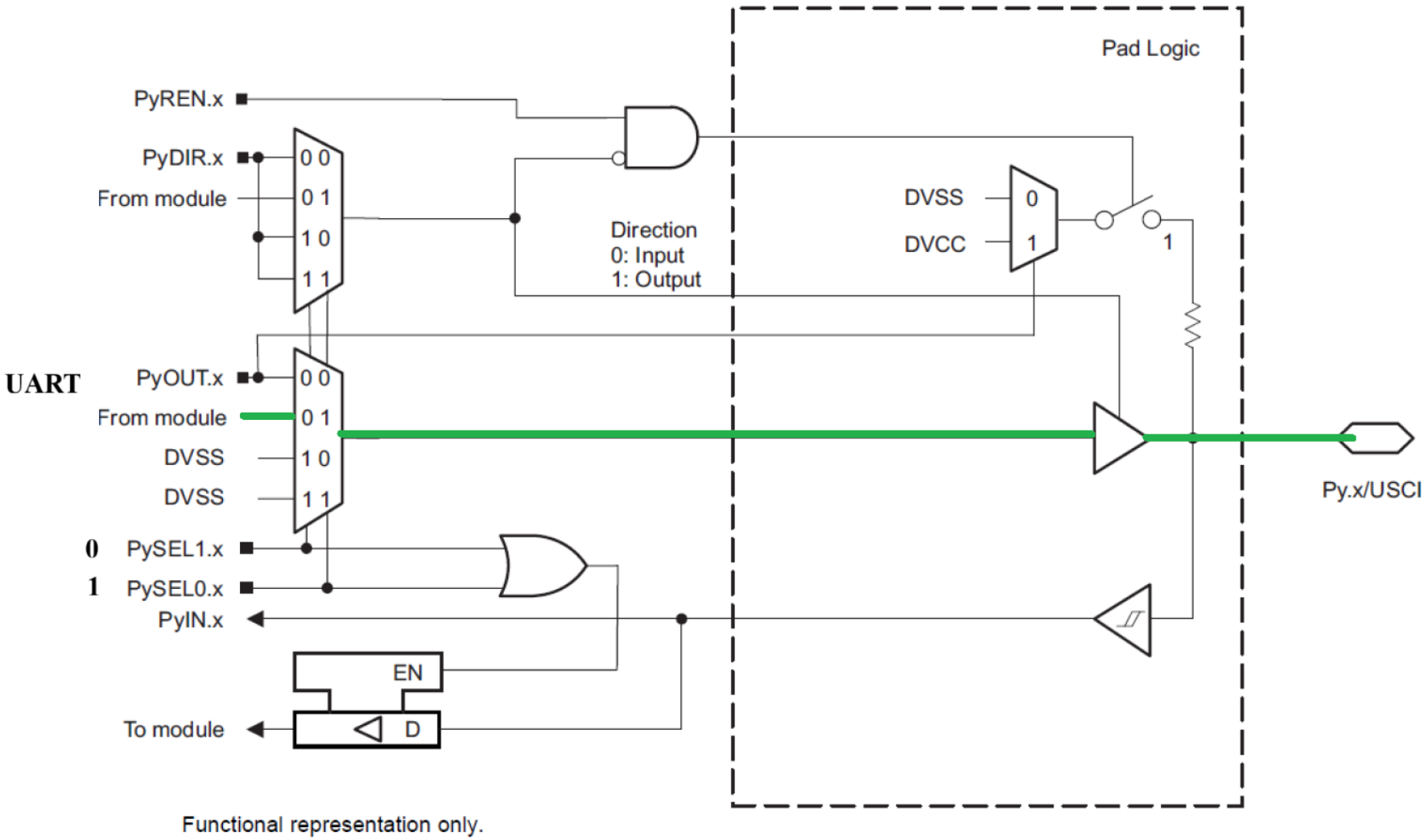
				SEL1	SEL0
P1.2/UCA0RXD/UCA0SOMI	2	P1.2 (I/O)	I: 0; O: 1	0	0
		UCA0RXD/UCA0SOMI	X(2)	0	1
		N/A	0	1	0
		DVSS	1		
		N/A	0	1	1
		DVSS	1		
P1.3/UCA0TXD/UCA0SIMO	3	P1.3 (I/O)	I: 0; O: 1	0	0
		UCA0TXD/UCA0SIMO	X(2)	0	1
		N/A	0	1	0
		DVSS	1		
		N/A	0	1	1
		DVSS	1		

Example: P1.2 UART Selected for Rx



Functional representation only.

Example: P1.3 UART Selected for Tx



Functional representation only.

Configuring P1.2 and P1.3 in Software

- Can use memory-mapped addresses (Yuck!!!)
- Can use DriverLib (Yes!!!)

```
GPIO_setAsPeripheralModuleFunctionInputPin(
            GPIO_PORT_P1,
            GPIO_PIN2,
            GPIO_PRIMARY_MODULE_FUNCTION);

GPIO_setAsPeripheralModuleFunctionOutputPin(
            GPIO_PORT_P1,
            GPIO_PIN3,
            GPIO_PRIMARY_MODULE_FUNCTION);
```

with

```
#define GPIO_PRIMARY_MODULE_FUNCTION (0x01)
#define GPIO_SECONDARY_MODULE_FUNCTION (0x02)
#define GPIO_TERTIARY_MODULE_FUNCTION (0x03)
```

Programming the UART

```
typedef struct _eUSCI_eUSCI_UART_ConfigV1
{
    uint_fast8_t selectClockSource;
    uint_fast16_t clockPrescalar;
    uint_fast8_t firstModReg;
    uint_fast8_t secondModReg;
    uint_fast8_t parity;
    unit_fast16_t msborLsbFirst;
    unit_fast16_t numberofStopBits;
    unit_fast16_t uartMode;
    unit_fast8_t oversampling;
    unit_fast16_t dataLength;
} eUSCI_UART_ConfigV1;
```

Configure UART Parameters

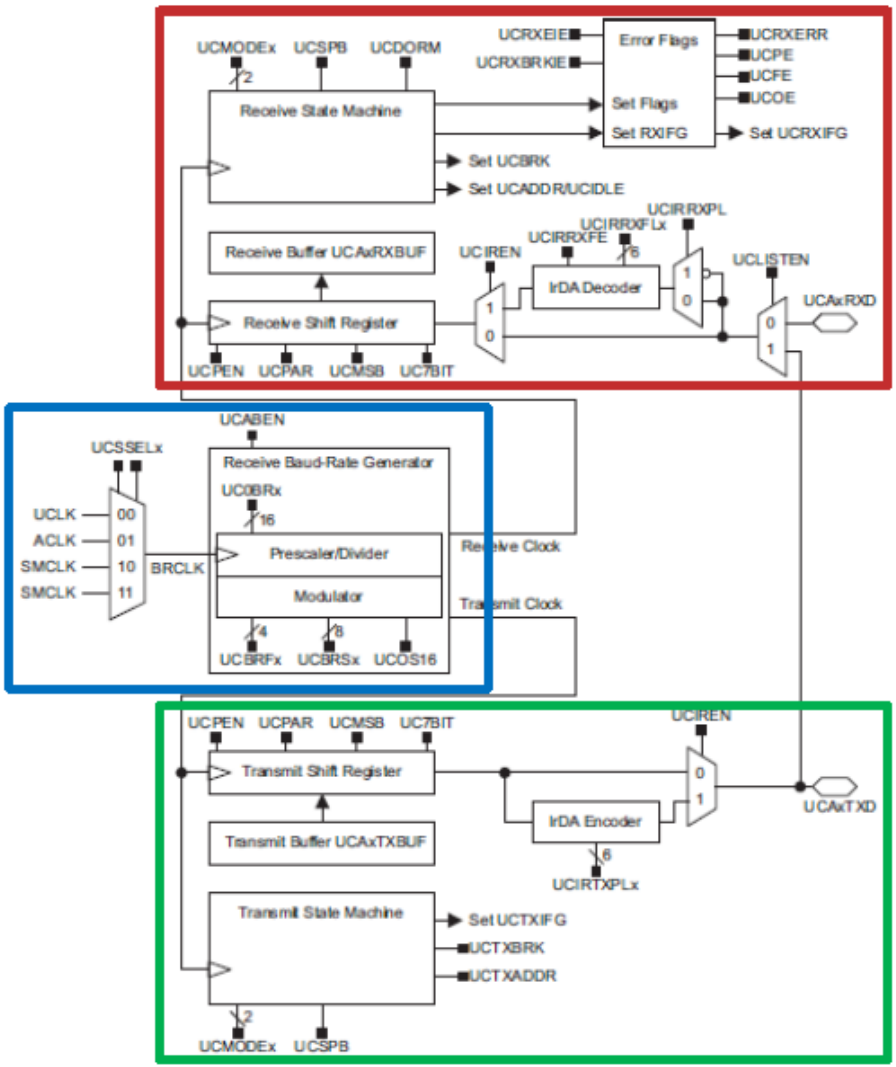
```
eUSCI_UART_ConfigV1 uartConfig;

// Related to baudrate generation and sampling
uartConfig.selectClockSource =  EUSCI_A_UART_CLOCKSOURCE_SMCLK;
uartConfig.clockPrescalar = 19;         // UCBR   = 19
uartConfig.firstModReg = 8;             // UCBRF  = 8
uartConfig.secondModReg = 0x55;         // UCBRS  = 0x55
uartConfig.overSampling = EUSCI_A_UART_OVERSAMPLING_BAUDRATE_GENERATION; // Oversampling

// Related to other configurations
uartConfig.parity = EUSCI_A_UART_NO_PARITY;                  // No praity
uartConfig.msborLsbFirst = EUSCI_A_UART_LSB_FIRST;           // LSB First
uartConfig.numberofStopBits = EUSCI_A_UART_ONE_STOP_BIT;     // One stop bit
uartConfig.dataLength = EUSCI_A_UART_8_BIT_LEN;              // Data length is 8 bits

// Beyond this course's knowledge base
uartConfig.uartMode = EUSCI_A_UART_MODE;             // UART mode -- the most typical usage
```

Hardware Functional Partitioning



- Top red frame: Rx Functionality
- Middle blue frame: Baudrate generation
- Bottom gGreen frame: Tx Functionality

Sampling Rate Calculation

- Calculate division factor $N = f_{\text{BRCKL}}/B$ (Note: clock frequency/baudrate)
- Expand N as $N = (16x + y) + \text{Frac}$
  - Where $x = \text{Int}(N/16), y = N\%16, \text{Frac} = N - \text{Int}(N)$
- pattern = lookup table (24-4) for Frac
- oversampling = $(N > 16)$ (1 or 0)

Sampling Rate Calculation

Symbol in the formulas	Name in Driverlib	MSP432 user manual name	Formula ($N > 16$)	Formula ($N \leq 16$)
B	N/A	Baudrate	N/A	same
f_{BRCLK}	N/A	BRCLK frequency	N/A	same
N	N/A	Division factor	f_{BRCLK}/B	same
x	clockPrescalar	UCBR	INT$(N/16)$	N
y	firstModReg	UCBRF	$N\%16$	0
Frac	N/A	N/A	$N - \text{Int}(N)$	same
pattern	secondModReg	UCBRS	Table 24-2(Frac)	same
oversampling	oversampling	OS16	1	0

Writing a UART Application: Highlights

```
// Configure Tx and Rx pins of EUSCI_A0 for UART
GPIO_setAsPeripheralModuleFunctionInputPin(
            GPIO_PORT_P1,
            GPIO_PIN2,
            GPIO_PRIMARY_MODULE_FUNCTION);

GPIO_setAsPeripheralModuleFunctionOutputPin(
            GPIO_PORT_P1,
            GPIO_PIN3,
            GPIO_PRIMARY_MODULE_FUNCTION);

// Create a UART configuration variable (and populate it)
eUSCI_UART_ConfigV1 uartConfig;
```

Writing a UART Application: Highlights

```
// initialize and enable EUSCI_A0 and set the baud rate
UART_initModule(EUSCI_A0_BASE, &uartConfig);
UART_enableModule(EUSCI_A0_BASE);

UARTBaudRate_t baudRate = baud9600;
```

What is Blocking?

```
// The following function waits until it has received a
// character
rChar = UART_receiveData(EUSCI_A0_BASE);

// We can check to see if a character is ready to be received
if (UART_getInterruptStatus (EUSCI_A0_BASE,
                  EUSCI_A_UART_RECEIVE_INTERRUPT_FLAG))
{
// and now we can receive the character withou blocking
rChar = UART_receiveData(EUSCI_A0_BASE);
}

// We must take similar action to transmit a character
if (UART_getInterruptStatus (EUSCI_A0_BASE,
                    EUSCI_A_UART_TRANSMIT_INTERRUPT_FLAG))
{
    UART_transmitData(EUSCI_A0_BASE, tChar);
}
```

Changing the Baud Rate

Must initialize and enable the UART after changing the baud rate

```
uartConfig.clockPrescalar = 9;
uartConfig.firstModReg = 12;
uartConfig.secondModReg = 0x22;
UART_initModule(EUSCI_A0_BASE, &uartConfig);
UART_enableModule(EUSCI_A0_BASE);
baudRate = baud19200;
```

Summary

- Practical introduction to serial communication using a UART
  - Connected MSP432 to our laptops using USB and MobaXterm
  - Studied example code that used DriverLib
  - Set up the UART hardware registers to transmit and receive
- UART block diagram and Port/Pin setup